Genetic Algorithm

in Artificial Intelligence

Code Optimization via Hyperparameter Tuning

Complete Guide — Algorithm + Dataset + Full Python Code

Python 3.x | **scikit-learn** | **GradientBoosting** | **Beginner Friendly**

1. What is a Genetic Algorithm?

A **Genetic Algorithm (GA)** is a search and optimization method inspired by **Darwin's Theory of Evolution**. The strongest solutions survive, reproduce, and pass on their traits to create even better solutions — generation after generation.

> ***Simple Real-Life Analogy:*** *Imagine finding the best cake recipe. Start with 30 random recipes (population). Taste each one (evaluate fitness). Mix ingredients from the two best recipes (crossover). Accidentally add a random ingredient (mutation). Keep the best recipes, discard the worst (selection). After 20 rounds of this process, you end up with a fantastic recipe!*

2. Real-World Applications of GA

Field	Application

Machine Learning	Tuning model hyperparameters automatically
Engineering	Designing airplane wings and structural components
Finance	Building profitable trading and investment strategies
Game AI	Evolving intelligent game-playing agents
Robotics	Optimizing robot movement and control systems
Medicine	Drug discovery, protein folding, treatment planning

3. Key Terminology

Chromosome	One complete solution. A list of 5 hyperparameter values e.g. [0.05, 100, 5, 3, 0.8]
Gene	A single value inside a chromosome. e.g. learning_rate = 0.05 is one gene.
Population	A group of chromosomes. In our code: 30 different sets of hyperparameters.
Fitness Function	The scoring method — 5-fold cross-validation accuracy. Higher score = better solution.
Selection	Choosing the best chromosomes to become parents for the next generation.
Crossover	Mixing two parent chromosomes to create child chromosomes (recombination).
Mutation	Randomly changing a gene slightly. Adds variety and prevents getting stuck.
Elitism	Always keeping the top N best chromosomes so the best solution is never lost.
Generation	One full cycle: evaluate all → select → crossover → mutate → new population.

4. Our Problem: Hyperparameter Optimization

We use GA to automatically find the best hyperparameters for a **Gradient Boosting Classifier**. Instead of manually trying combinations (which could take hours or days), the GA intelligently searches the space and finds a near-optimal solution in minutes.

The Dataset (Auto-Generated with scikit-learn)

```
from sklearn.datasets import make_classification

X, y = make_classification(
n_samples = 1000, # 1000 rows of data
n_features = 10, # 10 input columns (features)
n_informative = 6, # 6 columns genuinely matter
n_redundant = 2, # 2 columns are combinations of informative ones
n_classes = 2, # Binary task: predict 0 or 1
random_state = 42 # Fixed seed = same dataset every run
)
```

Property	Value	Meaning
n_samples	1000	Number of data rows
n_features	10	Number of input columns X
n_informative	6	Features that genuinely predict the label
n_redundant	2	Features derived from informative ones
n_classes	2	Binary classification: 0 or 1
random_state	42	Fixed seed for reproducibility

What the GA Optimizes (Search Space)

Parameter	Range	Type	Purpose
learning_rate	0.0001-0.5	float	How fast the model learns each step
n_estimators	10-300	int	Number of decision trees in the ensemble
max_depth	1-20	int	How deep each decision tree grows
min_samples_split	2-20	int	Min samples required to split a tree node
subsample	0.5-1.0	float	Fraction of training data used per tree

5. The 7 Steps of the Genetic Algorithm

1 **Chromosome Encoding**
Represent each solution as a list of 5 numbers: [learning_rate, n_estimators, max_depth, min_samples_split, subsample]. Example: [0.05, 100, 5, 3, 0.8]. This is one chromosome.

2 **Initialize Population**
Create 30 random chromosomes. Each has random values within allowed ranges. This is our starting pool of 30 candidate solutions to explore.

3 **Evaluate Fitness**
For each chromosome, train a GradientBoostingClassifier and run 5-fold cross-validation. Average accuracy = fitness score. Results are cached to avoid re-computing same parameters.

4 **Tournament Selection**
Pick 5 random chromosomes, return the one with highest accuracy as a parent. Repeat to get second parent. Better chromosomes win more often but all have a chance.

5 **Uniform Crossover**
For each gene position, flip a coin. Child inherits gene from parent1 (heads) or parent2 (tails). Two children are produced. Mixes best traits of both parents.

6 **Gaussian Mutation**
Each gene has 20% chance of being randomly changed. Floats get Gaussian noise. Integers get a random step added or subtracted. Adds variety and prevents getting stuck.

7 **Elitism + Replacement**
Top 4 chromosomes survive unchanged into next generation. Remaining 26 slots filled by new crossover+mutation children. Best solution is never lost.

6. Code Walkthrough — Section by Section

6.1 Imports & Dataset Setup

```
import random, numpy as np, matplotlib.pyplot as plt
import warnings; warnings.filterwarnings('ignore')
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import cross_val_score

SEED = 42
random.seed(SEED); np.random.seed(SEED)

X, y = make_classification(n_samples=1000, n_features=10,
n_informative=6, n_redundant=2,
n_classes=2, random_state=SEED)
```

We import all libraries, fix the random seed for reproducibility, and generate our classification dataset. scikit-learn builds the dataset, trains models, and runs cross-validation. matplotlib creates the results chart.

6.2 Hyperparameter Search Space

```
PARAM_SPACE = {
'learning_rate': (0.0001, 0.5), # float gene
'n_estimators': (10, 300), # integer gene
'max_depth': (1, 20), # integer gene
'min_samples_split': (2, 20), # integer gene
'subsample': (0.5, 1.0), # float gene
}
```

A dictionary defining valid ranges for each hyperparameter. The GA will only generate values within these bounds. The clamp() function enforces these limits after mutation.

6.3 Random Chromosome & Decode

```
def random_chromosome():
return [
round(random.uniform(0.0001, 0.5), 5), # learning_rate (float)
random.randint(10, 300), # n_estimators (int)
random.randint(1, 20), # max_depth (int)
random.randint(2, 20), # min_samples_split (int)
round(random.uniform(0.5, 1.0), 4), # subsample (float)
]

def decode(chromosome):
return {
'learning_rate': chromosome[0],
'n_estimators': int(chromosome[1]),
'max_depth': int(chromosome[2]),
'min_samples_split': int(chromosome[3]),
'subsample': chromosome[4],
}
```

random_chromosome() creates ONE random solution — a list of 5 values. decode() converts that list into a named dictionary so we can pass it directly to GradientBoostingClassifier using **params unpacking.

6.4 Fitness Function (The Core)

```
fitness_cache = {} # Dictionary to store already-computed results

def fitness(chromosome):
key = tuple(chromosome) # Convert list to hashable key
if key in fitness_cache: # Already computed? Return instantly
return fitness_cache[key]

params = decode(chromosome) # List -> named dict
model = GradientBoostingClassifier(**params, random_state=SEED)
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
acc = round(scores.mean(), 6) # Average accuracy across 5 folds
fitness_cache[key] = acc # Save result for future lookups
return acc
```

This is the most important function. For each chromosome (set of hyperparameters), we build and evaluate a model using 5-fold cross-validation — splitting data into 5 parts, training on 4, testing on 1, repeating 5 times. The cache avoids re-training identical models, saving significant computation time.

6.5 Tournament Selection

```
def tournament_selection(population, fitnesses, k=5):
indices = random.sample(range(len(population)), k) # Random 5 indices
best_idx = max(indices, key=lambda i: fitnesses[i]) # Highest fitness wins
return population[best_idx][:] # Return a COPY of the winner
```

Randomly picks 5 chromosomes and returns the one with highest accuracy. Better chromosomes win more often, but weaker ones still have a chance. This balances between using the best known solutions and exploring new ones. We return a copy ([:]) so mutations don't affect the original.

6.6 Uniform Crossover

```
def uniform_crossover(parent1, parent2):
child1, child2 = [], []
for g1, g2 in zip(parent1, parent2): # For each gene position
if random.random() < 0.5: # 50% coin flip
child1.append(g1); child2.append(g2) # Keep order
else:
child1.append(g2); child2.append(g1) # Swap
return child1, child2
```

For each gene, we flip a coin independently. Heads: child1 gets parent1's gene, child2 gets parent2's gene. Tails: they swap. Two children are created at once. Uniform crossover mixes solutions more thoroughly than single-point crossover.

6.7 Gaussian Mutation

```
def mutate(chromosome, mutation_rate=0.2):
mutated = chromosome[:] # Work on a copy
bounds = list(PARAM_SPACE.values())
for i in range(len(mutated)):
if random.random() < mutation_rate: # 20% chance per gene
lo, hi = bounds[i]
if i in [1, 2, 3]: # Integer genes
step = random.randint(1, max(1,(hi-lo)//5))
mutated[i] += random.choice([-step, step])
else: # Float genes
sigma = (hi - lo) * 0.1
mutated[i] += random.gauss(0, sigma)
return clamp(mutated) # Ensure all values stay in valid range
```

Each gene has a 20% probability of being randomly altered. Float genes receive Gaussian (bell-curve) noise — small changes more likely than large ones. Integer genes get a random step up or down. clamp() ensures mutated values stay within valid bounds. Mutation is crucial for exploring unseen areas of the search space.

7. The Main GA Loop — All Steps Combined

```
def genetic_algorithm(pop_size=30, generations=20,
mutation_rate=0.2, elite_size=4, tournament_k=5):

population = init_population(pop_size) # 30 random chromosomes
best_scores, avg_scores = [], []
overall_best, overall_best_score = None, 0.0

for gen in range(1, generations + 1): # 20 generations

fitnesses = [fitness(c) for c in population] # Evaluate all 30

gen_best = max(fitnesses)
best_scores.append(gen_best) # Track best per generation
avg_scores.append(sum(fitnesses)/len(fitnesses))

best_idx = fitnesses.index(gen_best)
if gen_best > overall_best_score: # Update all-time best
overall_best = population[best_idx][:]
overall_best_score = gen_best

# Elitism: keep top 4 unchanged
sorted_pop = sorted(zip(fitnesses, population),
key=lambda p: p[0], reverse=True)
new_population = [x for _, x in sorted_pop[:elite_size]]

# Fill rest: select -> crossover -> mutate
while len(new_population) < pop_size:
p1 = tournament_selection(population, fitnesses, tournament_k)
p2 = tournament_selection(population, fitnesses, tournament_k)
c1, c2 = uniform_crossover(p1, p2)
c1 = mutate(c1, mutation_rate)
c2 = mutate(c2, mutation_rate)
new_population.extend([c1, c2])

population = new_population[:pop_size] # New generation of 30

return overall_best, overall_best_score, best_scores, avg_scores
```

This loop is the complete GA in action. Each of the 20 generations: evaluates all 30 chromosomes, preserves the top 4 (elitism), then fills the remaining 26 slots by selecting parents, crossing them over, and mutating. The best solution found across ALL generations — not just the last — is returned.

8. Understanding the Output

```
Gen 1/20 | Best Acc: 0.8920 | Avg Acc: 0.8710 | lr=0.0523, n_est=187, depth=4
Gen 5/20 | Best Acc: 0.9040 | Avg Acc: 0.8830 | lr=0.0341, n_est=210, depth=5
Gen 10/20 | Best Acc: 0.9110 | Avg Acc: 0.8950 | lr=0.0285, n_est=245, depth=6
Gen 20/20 | Best Acc: 0.9180 | Avg Acc: 0.9050 | lr=0.0210, n_est=267, depth=7

BEST HYPERPARAMETERS FOUND:
learning_rate : 0.0210
n_estimators : 267
max_depth : 7
min_samples_split : 4
subsample : 0.872

Default Model Accuracy : 90.80 %
GA-Optimized Accuracy : 91.80 %
Improvement : +1.00 %
```

Output Field	What It Means
Best Acc	Highest cross-validation accuracy found in this generation
Avg Acc	Average accuracy across all 30 chromosomes — shows overall improvement
lr	The learning_rate of the best chromosome this generation
n_est	The n_estimators of the best chromosome this generation
depth	The max_depth of the best chromosome this generation
Improvement	Accuracy gain: GA-optimized model vs sklearn default settings

9. How to Run the Code

Step 1 — Install Required Libraries

```
pip install scikit-learn matplotlib numpy
```

Step 2 — Save & Run the Script

```
python genetic_algorithm_code_optimization.py
```

Step 3 — View Results

After running you will see:

1. *Generation-by-generation accuracy printed in the console*
2. *Best hyperparameters found printed clearly at the end*
3. *Default Model vs GA-Optimized accuracy comparison*
4. *A chart saved as **ga_optimization_results.png***

10. Tuning the GA Parameters

Parameter	Default	Increase Effect	Decrease Effect
pop_size	30	More diversity, slower per generation	Faster, may miss good solutions
generations	20	More improvement, longer runtime	Faster, less optimized result
mutation_rate	0.2	More exploration, less stable convergence	More stable, risk of getting stuck
elite_size	4	More stability, slower improvement	More diversity, may lose best
tournament_k	5	Higher selection pressure on best	More random, more exploration

11. Complete Algorithm Flow Summary

START	Create 30 random hyperparameter sets (initial population)
v	
EVALUATE	Train model with each chromosome, get 5-fold CV accuracy as fitness
v	
ELITE	Keep top 4 chromosomes unchanged into the next generation
v	
SELECT	Tournament: pick 5 random chromosomes, best one becomes a parent
v	

CROSSOVER	Combine 2 parents gene-by-gene with 50/50 coin flip per gene
v	
MUTATE	20% chance to randomly tweak each gene in the child chromosomes
v	
REPLACE	New generation = top 4 elites + 26 crossover+mutation children
v	
REPEAT	Go back to EVALUATE and run for 20 generations total
v	
OUTPUT	Return the single best chromosome found across all generations